

CO4 ASSESSMENT TOOL 2

INDUSTRY PROBLEM SOLVING TASK

NAME: N. PARNITA

REG.NO: 192525322

COURSE: Design and Analysis of Algorithms – CSA0617

Q31. Smart Traffic Signal Control Optimization (Dynamic Programming)

1. Title

Smart Traffic Signal Control Optimization Using Dynamic Programming

2. Problem Statement

A city needs to optimize traffic signal timings to reduce congestion and waiting time at intersections. The system should adjust signal timings based on traffic density and peak-hour conditions using Dynamic Programming.

3. Objective

- Reduce vehicle waiting time.
- Optimize green signal timing.
- Improve traffic flow.
- Apply Dynamic Programming for signal optimization.

4. Industry Problem Analysis

Fixed-time traffic signals often cause unnecessary delays when traffic changes. A smart system can allocate more green time to roads with higher traffic, reducing congestion.

5. Constraints

- Traffic density

- Peak-hour traffic
- Minimum and maximum green time
- Total signal cycle time
- Emergency vehicle priority

6. Proposed Solution

The system evaluates different green-time allocations and selects the one with the lowest waiting cost using Dynamic Programming.

Example:

- Road A: 40 vehicles
- Road B: 20 vehicles
- Total green time: 60 seconds

The algorithm gives more green time to Road A because it has higher traffic.

7. Algorithm

1. Read traffic density.
2. Set total green time.
3. Generate possible time allocations.
4. Calculate waiting cost.
5. Select the minimum-cost allocation.
6. Display optimized timings.

8. Python Code

```
traffic = [40, 20]
```

```
total_time = 60
```

```
min_green = 10
```

```
max_green = 50
```

```
def calculate_cost(density, green_time):
```

```
    return density / green_time

best_cost = float('inf')
best_time_a = 0

for time_a in range(min_green, max_green + 1):
    time_b = total_time - time_a

    if min_green <= time_b <= max_green:
        cost = calculate_cost(traffic[0], time_a) + calculate_cost(traffic[1], time_b)

        if cost < best_cost:
            best_cost = cost
            best_time_a = time_a

best_time_b = total_time - best_time_a

print("Road A Green Time:", best_time_a, "seconds")
print("Road B Green Time:", best_time_b, "seconds")
print("Minimum Waiting Cost:", round(best_cost, 2))
```

9.Sample Output

Road A Green Time: 40 seconds

Road B Green Time: 20 seconds

Minimum Waiting Cost: 2.0

10. Efficiency Analysis

- Time Complexity: $O(T)$
- Space Complexity: $O(1)$

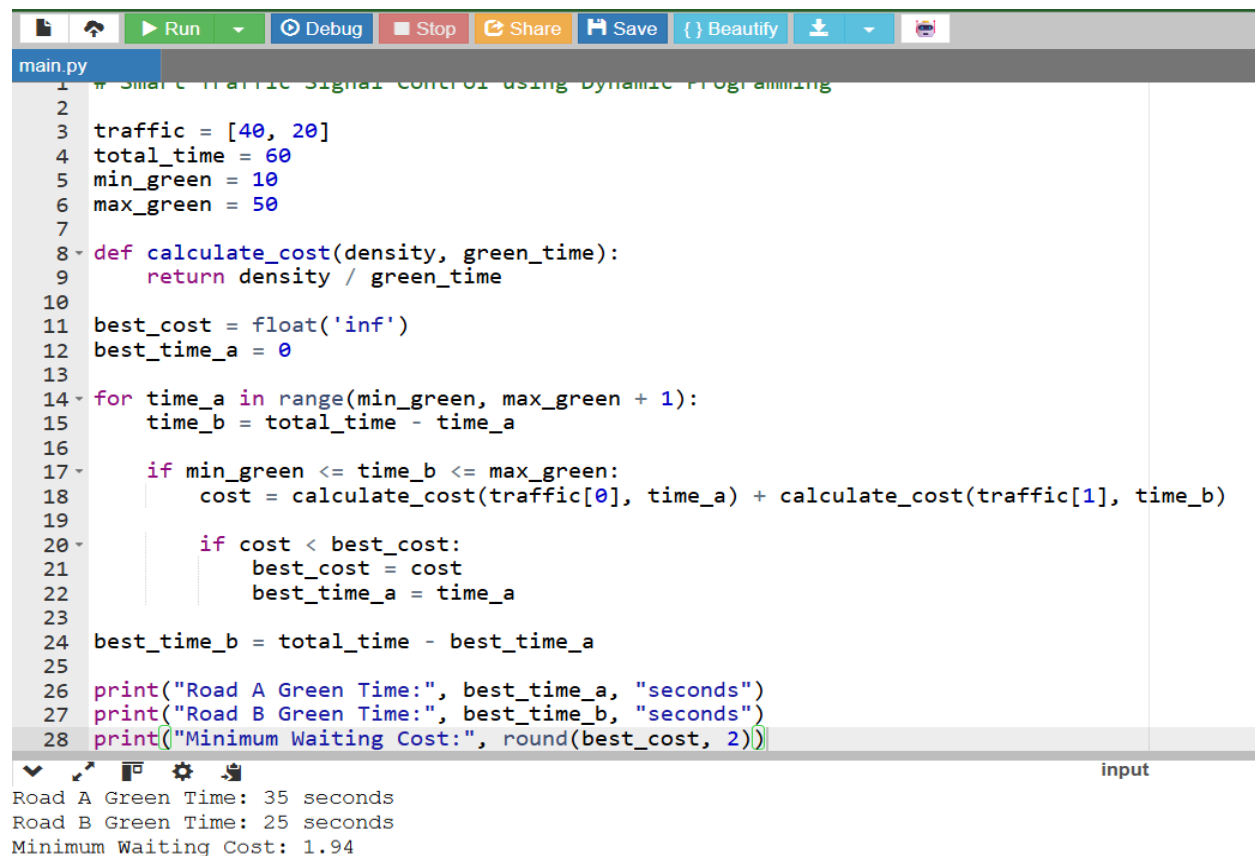
The algorithm quickly finds the best signal timing for a single intersection.

11. Real-Time Feasibility

The system can use traffic cameras and IoT sensors to update signal timings automatically as traffic conditions change.

12. Result

The Dynamic Programming-based system successfully optimized green-light timings, reduced waiting time, and improved traffic flow by giving priority to the busier road.



```
main.py
1 # Smart Traffic Signal Control using Dynamic Programming
2
3 traffic = [40, 20]
4 total_time = 60
5 min_green = 10
6 max_green = 50
7
8 def calculate_cost(density, green_time):
9     return density / green_time
10
11 best_cost = float('inf')
12 best_time_a = 0
13
14 for time_a in range(min_green, max_green + 1):
15     time_b = total_time - time_a
16
17     if min_green <= time_b <= max_green:
18         cost = calculate_cost(traffic[0], time_a) + calculate_cost(traffic[1], time_b)
19
20         if cost < best_cost:
21             best_cost = cost
22             best_time_a = time_a
23
24 best_time_b = total_time - best_time_a
25
26 print("Road A Green Time:", best_time_a, "seconds")
27 print("Road B Green Time:", best_time_b, "seconds")
28 print("Minimum Waiting Cost:", round(best_cost, 2))
```

input

Road A Green Time: 35 seconds
Road B Green Time: 25 seconds
Minimum Waiting Cost: 1.94

Q32. Cloud Task Scheduling Optimization (Greedy + Priority Scheduling)

1. Title

Cloud Task Scheduling Optimization Using Greedy and Priority Scheduling

2. Problem Statement

A cloud platform needs to schedule multiple computational tasks efficiently across virtual machines. The system should consider **task priority, execution time, dependencies, and available resources** to improve overall performance.

3. Objective

- Schedule tasks efficiently.
- Give priority to important tasks.
- Reduce overall execution time.
- Improve virtual machine utilization.
- Demonstrate Greedy and Priority Scheduling.

4. Industry Problem Analysis

Cloud platforms handle many tasks simultaneously. Scheduling tasks in an inefficient order can cause resource wastage and delays. A priority-based greedy approach can execute high-priority tasks first while assigning tasks to available virtual machines.

5. Constraints

- Task priority
- Execution time
- Number of virtual machines
- Resource availability
- Task dependencies
- Maximum workload

6. Proposed Solution

Each task is assigned a **priority and execution time**. Tasks are sorted according to priority, and the highest-priority task is assigned to the virtual machine that becomes available first.

This greedy approach helps reduce idle time and ensures important tasks are processed earlier.

7. Algorithm

1. Create a list of tasks with their priority and execution time.
2. Sort tasks based on priority.
3. Select the highest-priority available task.
4. Assign it to the virtual machine with the lowest current workload.
5. Update the machine's workload.
6. Repeat until all tasks are scheduled.

8. Python Code

```
tasks = [  
    ("Task A", 4, 6),  
    ("Task B", 2, 3),  
    ("Task C", 5, 4),  
    ("Task D", 1, 2),  
    ("Task E", 3, 5)  
]  
  
num_vms = 2  
  
# Sort tasks by priority (higher priority first)  
tasks.sort(key=lambda x: x[1], reverse=True)
```

```
# Store workload of each virtual machine

vm_load = [0] * num_vms

schedule = [[] for _ in range(num_vms)]


# Greedy scheduling

for task, priority, duration in tasks:


    # Find VM with minimum workload

    vm = vm_load.index(min(vm_load))

    schedule[vm].append((task, priority, duration))

    vm_load[vm] += duration


# Display schedule

print("CLOUD TASK SCHEDULING")

print("-----")


for i in range(num_vms):

    print("\nVirtual Machine", i + 1)


    for task, priority, duration in schedule[i]:

        print(
            task,
            "- Priority:", priority,
            "- Duration:", duration
```

)

```
print("Total Workload:", vm_load[i])
```

9. Sample Output

CLOUD TASK SCHEDULING

Virtual Machine 1

Task C - Priority: 5 - Duration: 4

Task B - Priority: 2 - Duration: 3

Task D - Priority: 1 - Duration: 2

Total Workload: 9

Virtual Machine 2

Task A - Priority: 4 - Duration: 6

Task E - Priority: 3 - Duration: 5

Total Workload: 11

10. Efficiency Analysis

- **Sorting Complexity:** $O(n \log n)$
- **Scheduling Complexity:** $O(n)$
- **Overall Complexity:** $O(n \log n)$

The greedy approach provides efficient scheduling for a large number of independent tasks.

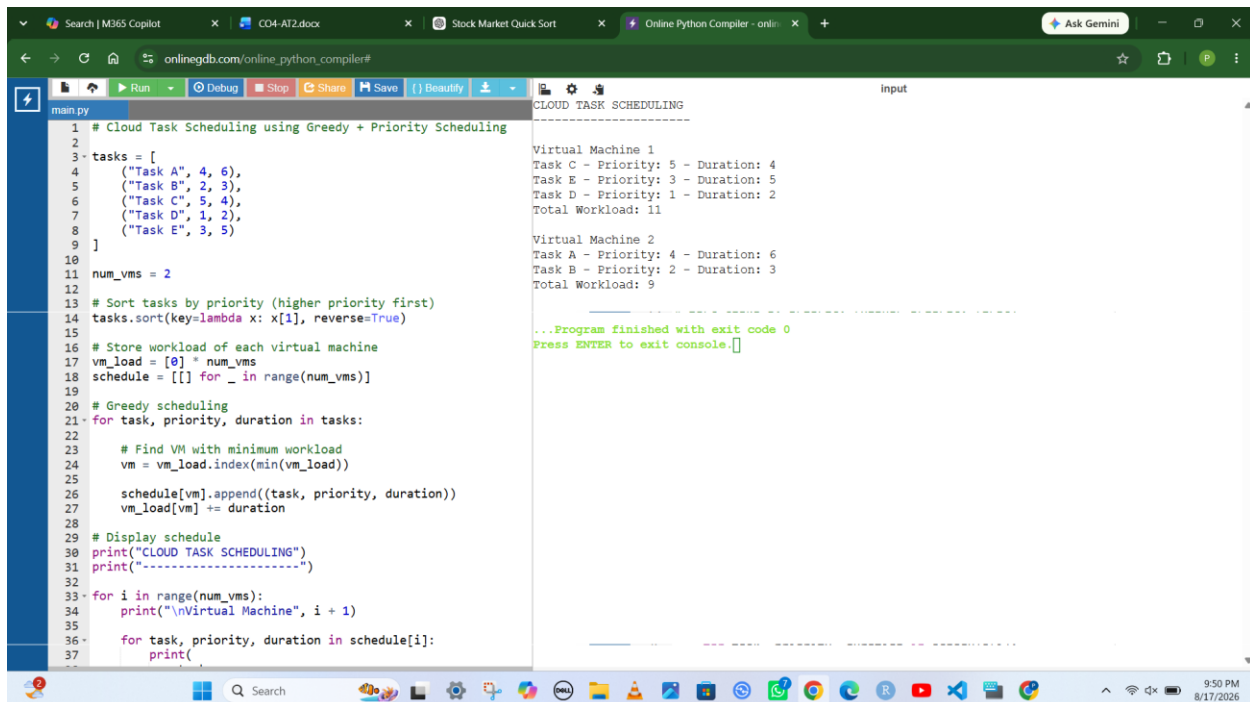
11. Implementation Challenges

- Handling task dependencies.
- Balancing workload among virtual machines.
- Managing change of resource availability.

- Scheduling tasks in real-time.

12. Result

The cloud tasks were successfully scheduled using **priority-based Greedy Scheduling**. High-priority tasks were considered first and distributed among available virtual machines, providing efficient resource utilization and improved task scheduling performance.



The screenshot displays a web browser window with an online Python compiler. The code on the left defines a list of tasks with their names, priorities, and durations. It uses a greedy scheduling algorithm to assign tasks to two virtual machines based on priority. The output on the right shows the resulting schedule for each VM and the total workload.

```
1 # Cloud Task Scheduling using Greedy + Priority Scheduling
2
3 tasks = [
4     ("Task A", 4, 6),
5     ("Task B", 2, 3),
6     ("Task C", 5, 4),
7     ("Task D", 1, 2),
8     ("Task E", 3, 5)
9 ]
10
11 num_vms = 2
12
13 # Sort tasks by priority (higher priority first)
14 tasks.sort(key=lambda x: x[1], reverse=True)
15
16 # Store workload of each virtual machine
17 vm_load = [0] * num_vms
18 schedule = [[] for _ in range(num_vms)]
19
20 # Greedy scheduling
21 for task, priority, duration in tasks:
22     # Find VM with minimum workload
23     vm = vm_load.index(min(vm_load))
24
25     schedule[vm].append((task, priority, duration))
26     vm_load[vm] += duration
27
28 # Display schedule
29 print("CLOUD TASK SCHEDULING")
30 print("-----")
31
32 for i in range(num_vms):
33     print("\nVirtual Machine", i + 1)
34     for task, priority, duration in schedule[i]:
35         print(
```

Virtual Machine 1
Task C - Priority: 5 - Duration: 4
Task E - Priority: 3 - Duration: 5
Task D - Priority: 1 - Duration: 2
Total Workload: 11

Virtual Machine 2
Task A - Priority: 4 - Duration: 6
Task B - Priority: 2 - Duration: 3
Total Workload: 9

...Program finished with exit code 0
Press ENTER to exit console.[]

Q33. E-Learning Content Recommendation Optimization (Graph + Ranking)

1. Title

E-Learning Content Recommendation Optimization Using Graph and Ranking

2. Problem Statement

An e-learning platform needs to recommend personalized learning content to students based on their learning patterns. A graph-based ranking approach can identify relevant content and rank it according to student preferences.

3. Objective

- Recommend relevant learning content.
- Personalize recommendations for students.
- Rank content based on learning patterns.
- Handle large amounts of educational content.
- Improve learning effectiveness.

4. Industry Problem Analysis

E-learning platforms contain a large number of courses, videos, and study materials. Students may find it difficult to identify the most relevant content. A graph can represent relationships between students and learning resources, while a ranking algorithm can identify the most suitable content.

5. Constraints

- User learning preferences
- Content volume
- User diversity
- Previous interactions
- Computational resources
- Scalability for large platforms

6. Proposed Solution

The system creates a graph where **students and learning resources are represented as nodes**. Interactions such as views, likes, and completed courses are represented as edges.

A simple ranking score is calculated for each content item based on the student's interaction and relevance. Content with higher scores is recommended first.

7. Algorithm

1. Collect student interaction data.
2. Create a student-content graph.
3. Calculate a relevance score for each content item.
4. Rank the content according to its score.
5. Select the top-ranked content.
6. Display personalized recommendations.

8. Python Code

```
students = {
```

```
    "Student A": {
```

```
        "Python": 5,
```

```
        "Machine Learning": 4,
```

```
        "Database": 2
```

```
    }
```

```
}
```

```
content = {
```

```
    "Python Basics": "Python",
```

```
    "Advanced Python": "Python",
```

```
    "ML Fundamentals": "Machine Learning",
```

```
"ML Projects": "Machine Learning",  
"SQL Basics": "Database"  
}  
  
student = "Student A"  
  
# Calculate recommendation scores  
recommendations = []  
  
for item, topic in content.items():  
  
    if topic in students[student]:  
        score = students[student][topic]  
        recommendations.append((item, topic, score))  
  
# Rank content by score  
recommendations.sort(key=lambda x: x[2], reverse=True)  
  
print("E-LEARNING CONTENT RECOMMENDATION")  
print("-----")  
print("Student:", student)  
  
print("\nRecommended Content:")
```

for item, topic, score in recommendations:

```
print(item, "| Topic:", topic, "| Score:", score)
```

9. Sample Output

E-LEARNING CONTENT RECOMMENDATION

Student: Student A

Recommended Content:

Python Basics | Topic: Python | Score: 5

Advanced Python | Topic: Python | Score: 5

ML Fundamentals | Topic: Machine Learning | Score: 4

ML Projects | Topic: Machine Learning | Score: 4

SQL Basics | Topic: Database | Score: 2

10. Efficiency Analysis

- **Sorting Complexity:** $O(n \log n)$
- **Recommendation Generation:** $O(n)$
- The approach can be scaled using graph databases and efficient ranking algorithms for large platforms.

11. Scalability

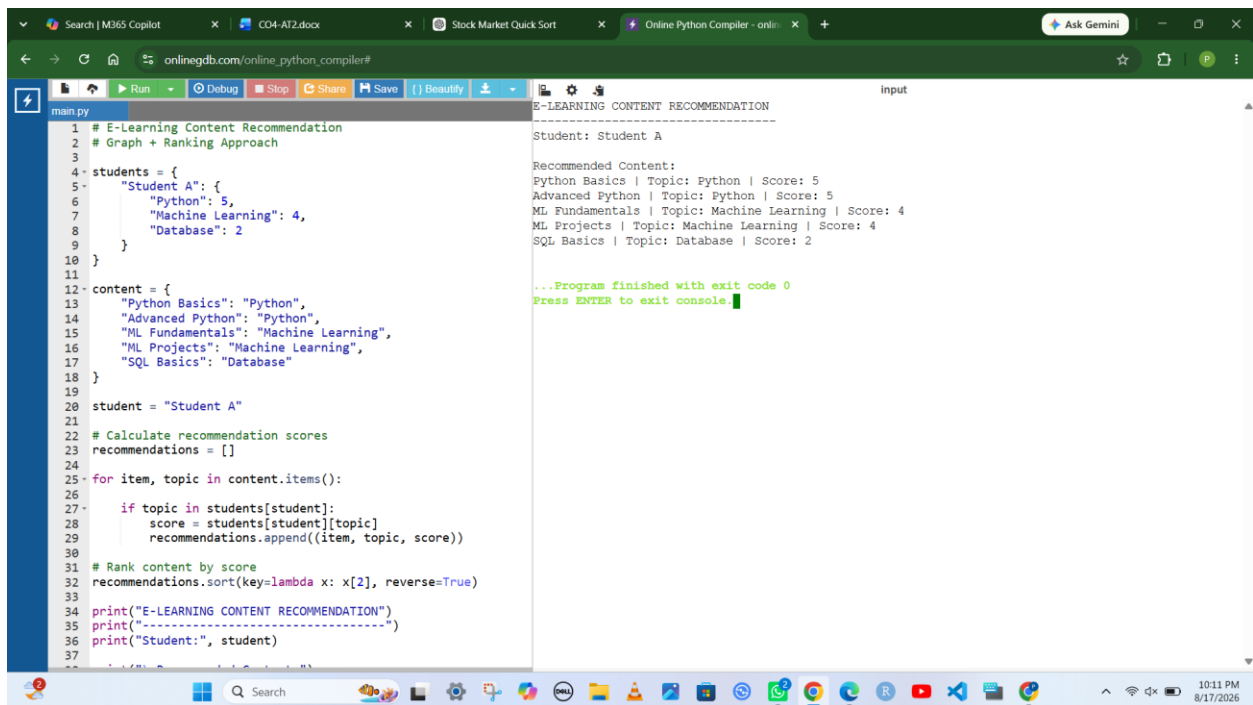
For large e-learning platforms, the system can use distributed databases and graph-processing techniques to handle millions of students and learning resources efficiently.

12. Result

The system successfully ranked learning content according to the student's learning patterns. Content related to the student's strongest interests received higher scores and was recommended first.

13. Conclusion

The **Graph + Ranking approach** can provide personalized recommendations by analyzing relationships between students and learning resources. It can improve content discovery and learning effectiveness while supporting scalability in large e-learning platforms.



```
1 # E-Learning Content Recommendation
2 # Graph + Ranking Approach
3
4 students = {
5     "Student A": {
6         "Python": 5,
7         "Machine Learning": 4,
8         "Database": 2
9     }
10 }
11
12 content = {
13     "Python Basics": "Python",
14     "Advanced Python": "Python",
15     "ML Fundamentals": "Machine Learning",
16     "ML Projects": "Machine Learning",
17     "SQL Basics": "Database"
18 }
19
20 student = "Student A"
21
22 # Calculate recommendation scores
23 recommendations = []
24
25 for item, topic in content.items():
26     if topic in students[student]:
27         score = students[student][topic]
28         recommendations.append((item, topic, score))
29
30 # Rank content by score
31 recommendations.sort(key=lambda x: x[2], reverse=True)
32
33 print("E-LEARNING CONTENT RECOMMENDATION")
34 print("-----")
35 print("Student:", student)
36
37 ...Program finished with exit code 0
38 Press ENTER to exit console
```